##Q1

In [4]:

```
print("1. Cleaning")
print("2. Normalization")
print("3. Integration")

ch = int(input("Enter choice: "))

if ch == 1:
    a = input("Enter data: ").split()
    a = [x for x in a if x.upper() != "NA"]
    print("Cleaned data:", a)

elif ch == 2:
    a = list(map(float, input("Enter data: ").split()))
    mn, mx = min(a), max(a)
    b = [(x-mn)/(mx-mn) for x in a]
    print("Normalized data:", b)

elif ch == 3:
    a = input("Enter first data: ").split()
    b = input("Enter second data: ").split()
    print("Integrated data:", a + b)

else:
    print("Invalid choice")
```

```
1. Cleaning
2. Normalization
3. Integration
Enter choice:  1
Enter data:  1 2 3 Na 4
Cleaned data: ['1', '2', '3', '4']
```

Q2

In [16]:

```
print("1. Horizontal")
print("2. Vertical")
print("3. Round Robin")
print("4. Hash Based")

ch = int(input("Enter choice: "))

if ch == 1:
    r = int(input("Enter number of rows: "))
    a = [input("Enter row: ").split() for _ in range(r)]
    n = int(input("Enter number of partitions: "))

    for i in range(n):
        print("Partition", i+1, ":", a[i::n])

elif ch == 2:
    r = int(input("Enter number of rows: "))
    a = [input("Enter row: ").split() for _ in range(r)]
    n = int(input("Split after column: "))

    print("Partition 1:", [row[:n] for row in a])
    print("Partition 2:", [row[n:] for row in a])

elif ch == 3:
    a = input("Enter data: ").split()
    n = int(input("Enter number of partitions: "))

    for i in range(n):
        print("Partition", i+1, ":", a[i::n])

elif ch == 4:
    a = input("Enter data: ").split()
    n = int(input("Enter number of partitions: "))

    for x in a:
        print(x, "-> Partition", hash(x) % n + 1)
```

```
else:
    print("Invalid choice")
```

```
1. Horizontal
2. Vertical
3. Round Robin
4. Hash Based
Enter choice:  4
Enter data:  1 2 3 4 5 6 7 8 9
Enter number of partitions:  3
1 -> Partition 3
2 -> Partition 2
3 -> Partition 1
4 -> Partition 2
5 -> Partition 1
6 -> Partition 3
7 -> Partition 2
8 -> Partition 1
9 -> Partition 1
```

Q3.

In [3]:

```
print("Schemas: 1. Star  2. Snowflake  3. Fact Constellation")
ch = int(input("Enter choice (1-3): "))

sales = input("Enter Sales Amount: ")
city = input("Enter City: ")

if ch == 1:
    # DIAGRAM:  [Dimension] <---> [FACT TABLE] <---> [Dimension]
    # (Direct connection to center)
    fact = {"Sales": sales, "Loc_ID": 101}
    dim_loc = {"Loc_ID": 101, "City": city}
    print("Star Schema:", fact, dim_loc)

elif ch == 2:
    # DIAGRAM:  [Sub-Dimension] <---> [Dimension] <---> [FACT TABLE]
    # (Branching out further like a snowflake)
    region = input("Enter Region: ")
    fact = {"Sales": sales, "Loc_ID": 101}
    dim_loc = {"Loc_ID": 101, "Reg_ID": 55}
    sub_dim = {"Reg_ID": 55, "Region": region}
    print("Snowflake:", fact, dim_loc, sub_dim)

elif ch == 3:
    # DIAGRAM:  [FACT 1] <---> [Shared Dimension] <---> [FACT 2]
    # (Multiple centers sharing tables)
    target = input("Enter Target Sales: ")
    fact_sales = {"Sales": sales, "Loc_ID": 101}
    fact_target = {"Target": target, "Loc_ID": 101}
    shared_dim = {"Loc_ID": 101, "City": city}
    print("Constellation:", fact_sales, shared_dim, fact_target)

else:
    print("Invalid Schema Choice.")
```

```
Schemas: 1. Star  2. Snowflake  3. Fact Constellation
Enter choice (1-3):  2
Enter Sales Amount:  2500
Enter City:  hyd
Enter Region:  india
Snowflake: {'Sales': '2500', 'Loc_ID': 101} {'Loc_ID': 101, 'Reg_ID': 55} {'Reg_ID': 55, 'Region': 'india'}
```

Q4

In [6]:

```
n = int(input("Enter number of records: "))

data = []
for i in range(n):
    data.append(input("Enter Product Month Sales: ").split())
```

```
print("\n1. Roll-up")
print("2. Drill-down")
print("3. Slice")
print("4. Dice")

ch = int(input("Enter choice: "))

if ch == 1:
    print("Total Sales:", sum(int(x[2]) for x in data))

elif ch == 2:
    p = input("Enter product: ")
    for x in data:
        if x[0] == p:
            print(x)

elif ch == 3:
    m = input("Enter month: ")
    for x in data:
        if x[1] == m:
            print(x)

elif ch == 4:
    p = input("Enter product: ")
    m = input("Enter month: ")
    for x in data:
        if x[0] == p and x[1] == m:
            print(x)

else:
    print("Invalid choice")
```

```
Enter number of records:  2
Enter Product Month Sales:  A B 1200
Enter Product Month Sales:  C D 100


1. Roll-up
2. Drill-down
3. Slice
4. Dice
Enter choice:  1
Total Sales: 1300
```

Q5

In [7]:

```
print("========================================")
print("      Data ETL Simulator System         ")
print("========================================")

# 1. EXTRACT PHASE:
# Getting raw input from the user and splitting it into a list
raw_text = input("Enter raw data (include 'NA' for missing): ")
data_extracted = raw_text.split()
print("\n[Status] --> Extraction Complete")
print("Extracted Data   :", data_extracted)

# 2. TRANSFORM PHASE:
# Using list comprehension to filter out any 'NA' values
print("\n[Status] --> Running Transformation...")
data_transformed = [x for x in data_extracted if x.upper() != "NA"]
print("Transformed Data :", data_transformed)

# 3. LOAD PHASE:
# Moving the cleaned data into the final target destination
print("\n[Status] --> Loading Data into Target DB...")
final_warehouse = data_transformed
print("Final Loaded Data:", final_warehouse)

print("========================================")
print("          ETL Process Finished          ")
```

```
========================================
      Data ETL Simulator System
```

```
========================================
Enter raw data (include 'NA' for missing):  1 2 3 Na

[Status] --> Extraction Complete
Extracted Data   : ['1', '2', '3', 'Na']

[Status] --> Running Transformation...
Transformed Data : ['1', '2', '3']

[Status] --> Loading Data into Target DB...
Final Loaded Data: ['1', '2', '3']
========================================
          ETL Process Finished
```

Q6

In [8]:

```
print("========================================")
print("  Attribute Oriented Induction (AOI)     ")
print("========================================")

n = int(input("Enter number of records to process: "))
data = []

# 1. INITIAL DATA COLLECTION
print("\n[Enter data format -> Name Age City]")
for i in range(n):
    name, age, city = input(f"Record {i+1}: ").split()
    data.append([name, int(age), city])

print("\n[Status] Applying Concept Hierarchies...")
# Hierarchy Rules:
# Age: < 25 becomes 'Young', else 'Adult'
# City: Hyderabad/Chennai becomes 'South', else 'Other'

print("\n--- Final Generalized Data ---")

# 2. GENERALIZATION PHASE
for x in data:
    n_val = x[0]
    a_val = x[1]
    c_val = x[2]

    # Applying the generalization rules
    age_group = "Young" if a_val < 25 else "Adult"
    city_group = "South" if c_val in ["Hyderabad", "Chennai"] else "Other"

    print(f"{n_val} --> {age_group} | {city_group}")

print("========================================")
```

```
========================================
  Attribute Oriented Induction (AOI)
========================================
Enter number of records to process:  3

[Enter data format -> Name Age City]
Record 1:  A 2 Hyd
Record 2:  B 22 Pune
Record 3:  Bow 30 delhi

[Status] Applying Concept Hierarchies...

--- Final Generalized Data ---
A --> Young | Other
B --> Young | Other
Bow --> Adult | Other
========================================
```

Q7

In [9]:

```
n = int(input("Enter number of transactions: "))

data = []
for i in range(n):
    data.append(input("Enter items: ").split())

minsup = int(input("Enter minimum support: "))

count = {}

for t in data:
    for item in t:
        count[item] = count.get(item, 0) + 1

print("APRIORI Frequent items:")
for item in count:
    if count[item] >= minsup:
        print(item, ":", count[item])
```

```
Enter number of transactions:  3
Enter items:  A B C
Enter items:  B D
Enter items:  D C
Enter minimum support:  2
APRIORI Frequent items:
B : 2
C : 2
D : 2
```

Q8

In [10]:

```
print("========================================")
print("           FP-Growth Algorithm          ")
print("========================================")

n = int(input("Enter number of transactions: "))
data = []
for i in range(n):
    data.append(input("Enter items: ").split())

minsup = int(input("Enter minimum support: "))

# 1. Count frequencies (Same as Apriori)
count = {}
for t in data:
    for item in t:
        count[item] = count.get(item, 0) + 1

# 2. Sort frequent items in descending order of frequency
freq = {k: v for k, v in count.items() if v >= minsup}
# Sorts items based on their count, highest to lowest
order = sorted(freq.keys(), key=lambda x: freq[x], reverse=True)

print("\nFrequent Items Ordered:", order)
print("\n--- Ordered Paths for FP-Tree ---")

# 3. Generate the sorted paths for each transaction
for t in data:
    # Keeps only frequent items and puts them in the correct order
    path = [x for x in order if x in t]
    if len(path) > 0:
        print("Path ->", path)
```

```
========================================
           FP-Growth Algorithm
========================================
Enter number of transactions:  3
Enter items:  A B C
Enter items:  B C
Enter items:  C A
Enter minimum support:  2

Frequent Items Ordered: ['C', 'A', 'B']
```

```
--- Ordered Paths for FP-Tree ---
Path -> ['C', 'A', 'B']
Path -> ['C', 'B']
Path -> ['C', 'A']
```

Q9

In [3]:

```
from sklearn.tree import DecisionTreeClassifier

n = int(input("Enter number of records: "))

X = []
y = []

for i in range(n):
    a = list(map(int, input("RunnyNose Cough Temperature: ").split()))
    X.append(a)
    y.append(input("Disease (Flu/Fever): "))

model = DecisionTreeClassifier()
model.fit(X, y)

a = list(map(int, input("Enter new patient data: ").split()))
print("Prediction:", model.predict([a])[0])
```

```
Enter number of records:  3
RunnyNose Cough Temperature:  1 0 1
Disease (Flu/Fever):  fever
RunnyNose Cough Temperature:  1 1 0
Disease (Flu/Fever):  cold
RunnyNose Cough Temperature:  1 0 0
Disease (Flu/Fever):  cold
Enter new patient data:  1 1 1
Prediction: cold
```

##Q2

Q10

In [7]:

```
from math import log2

a = input("Enter attribute values: ").split()
y = input("Enter target values: ").split()

def entropy(x):
    n = len(x)
    return -sum((x.count(v)/n)*log2(x.count(v)/n) for v in set(x))

e = entropy(y)
we = sum((a.count(v)/len(a))*entropy([y[i] for i in range(len(y)) if a[i] == v]) for v in set(a))

print("Entropy:", round(e,3))
print("Weighted Entropy:", round(we,3))
print("Information Gain:", round(e-we,3))
```

```
Enter attribute values:  sunny sunny sunny rain
Enter target values:  no no yes yes
Entropy: 1.0
Weighted Entropy: 0.689
Information Gain: 0.311
```

Q11

In [6]:

```
from sklearn.naive_bayes import GaussianNB

print("========================================")
print("  Student Success Predictor (Naive Bayes)")
print("========================================")
```

```
# 1. Training data: [Study Hours, Sleep Hours]
X = [[1, 5], [2, 6], [3, 7], [7, 8], [8, 7], [9, 8]]
y = [0, 0, 0, 1, 1, 1]  # 0 = Fail, 1 = Pass

# 2. Train model
model = GaussianNB()
model.fit(X, y)

# 3. User input
study = float(input("Enter study hours: "))
sleep = float(input("Enter sleep hours: "))

# 4. Prediction & Probabilities
result = model.predict([[study, sleep]])
probs = model.predict_proba([[study, sleep]])

print(f"\n[Status] Probability [Fail, Pass]: {probs[0]}")

# 5. Final Output
if result[0] == 1:
    print("---> Final Result: Student will Pass")
else:
    print("---> Final Result: Student will Fail")
print("========================================")
```

```
========================================
  Student Success Predictor (Naive Bayes)
========================================
Enter study hours:  8
Enter sleep hours:  8

[Status] Probability [Fail, Pass]: [6.93711232e-14 1.00000000e+00]
---> Final Result: Student will Pass
========================================
```

Q12

In [1]:

```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

# Training data
X = [[1, 2], [2, 3], [3, 3], [6, 7], [7, 8], [8, 8]]
y = ['Fail', 'Fail', 'Fail', 'Pass', 'Pass', 'Pass']

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

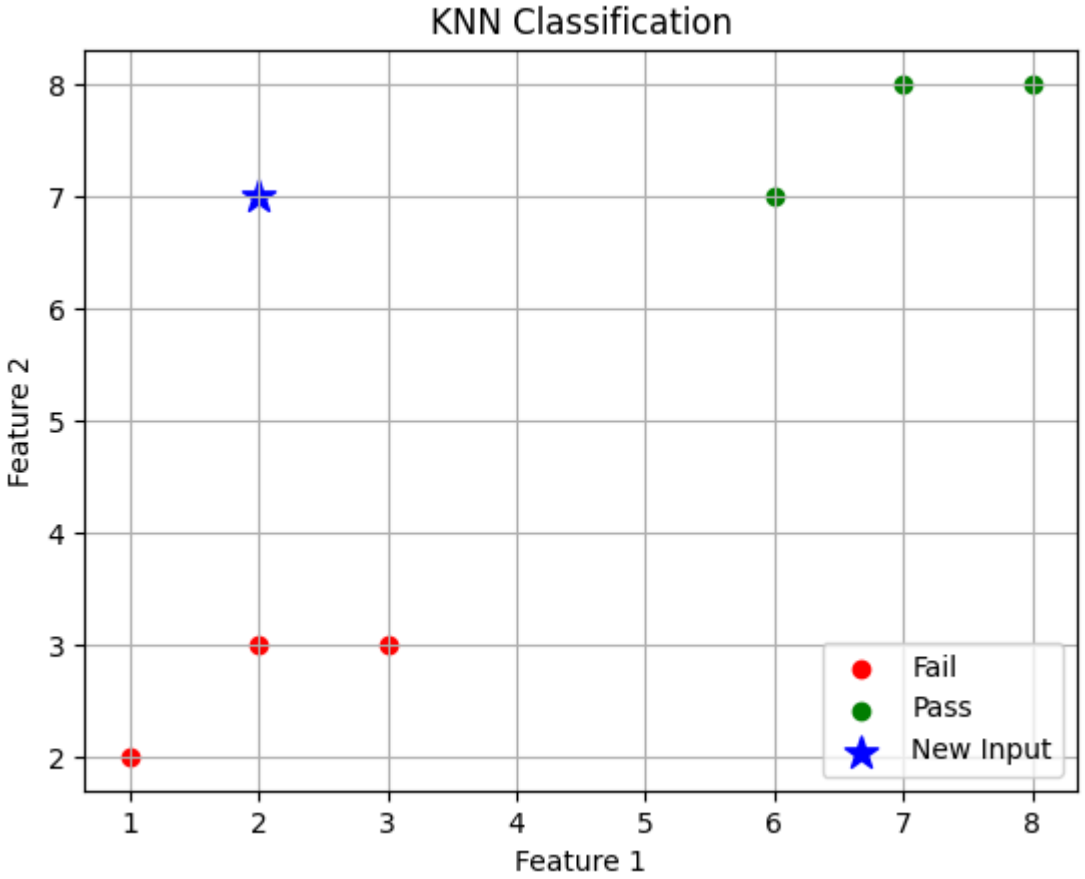
# User input
a = float(input("Enter first value: "))
b = float(input("Enter second value: "))

# Prediction
result = model.predict([[a, b]])
print("Predicted Class:", result[0])

# Scatter plot
plt.scatter([1,2,3], [2,3,3], color='red', label='Fail')
plt.scatter([6,7,8], [7,8,8], color='green', label='Pass')
plt.scatter(a, b, color='blue', marker='*', s=150, label='New Input')

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("KNN Classification")
plt.legend()
plt.grid()
plt.show()
```

```
Enter first value:  2
Enter second value:  7
Predicted Class: Fail
```



In [ ]:

```
Q13
```

In [2]:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Training data
X = [[1, 2], [2, 3], [3, 3], [8, 8], [9, 8], [8, 9]]

# User input
a = float(input("Enter first value: "))
b = float(input("Enter second value: "))

# Create and train model
model = KMeans(n_clusters=2, random_state=0, n_init=10)
model.fit(X)

# Predict cluster
result = model.predict([[a, b]])
print("Cluster:", result[0])

# Scatter plot
labels = model.labels_
plt.scatter([x[0] for x in X], [x[1] for x in X],
            c=labels, cmap='viridis', label='Clusters')

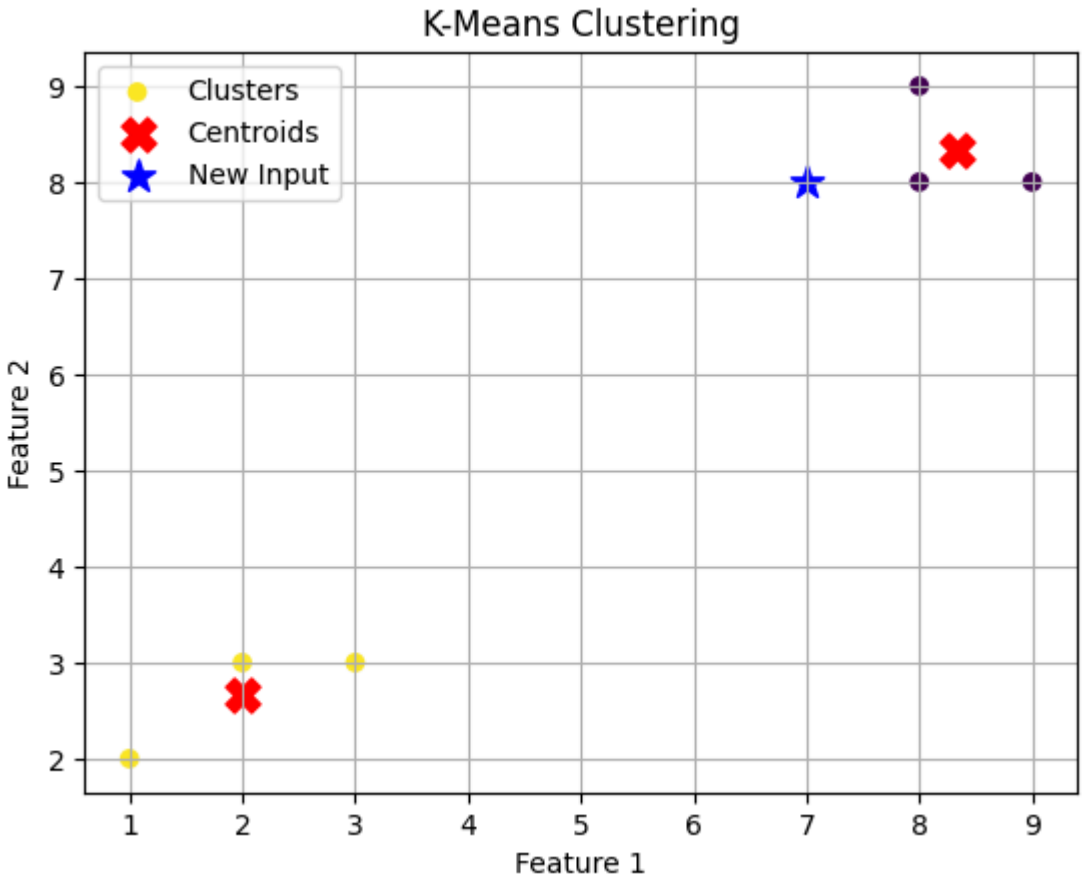
# Plot centroids
plt.scatter(model.cluster_centers_[:,0],
            model.cluster_centers_[:,1],
            color='red', marker='X', s=150, label='Centroids')

# Plot user input
plt.scatter(a, b, color='blue', marker='*', s=150, label='New Input')

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering")
plt.legend()
plt.grid()
plt.show()
```

```
Enter first value:  7
Enter second value:  8
```

```
Cluster: 0
```



Q14

In [3]:

```python
from sklearn.cluster import Birch
import matplotlib.pyplot as plt

# Training data
X = [[1, 2], [2, 3], [3, 3], [8, 8], [9, 8], [8, 9]]

# Create BIRCH model
model = Birch(n_clusters=2)
model.fit(X)

# User input
a = float(input("Enter first value: "))
b = float(input("Enter second value: "))

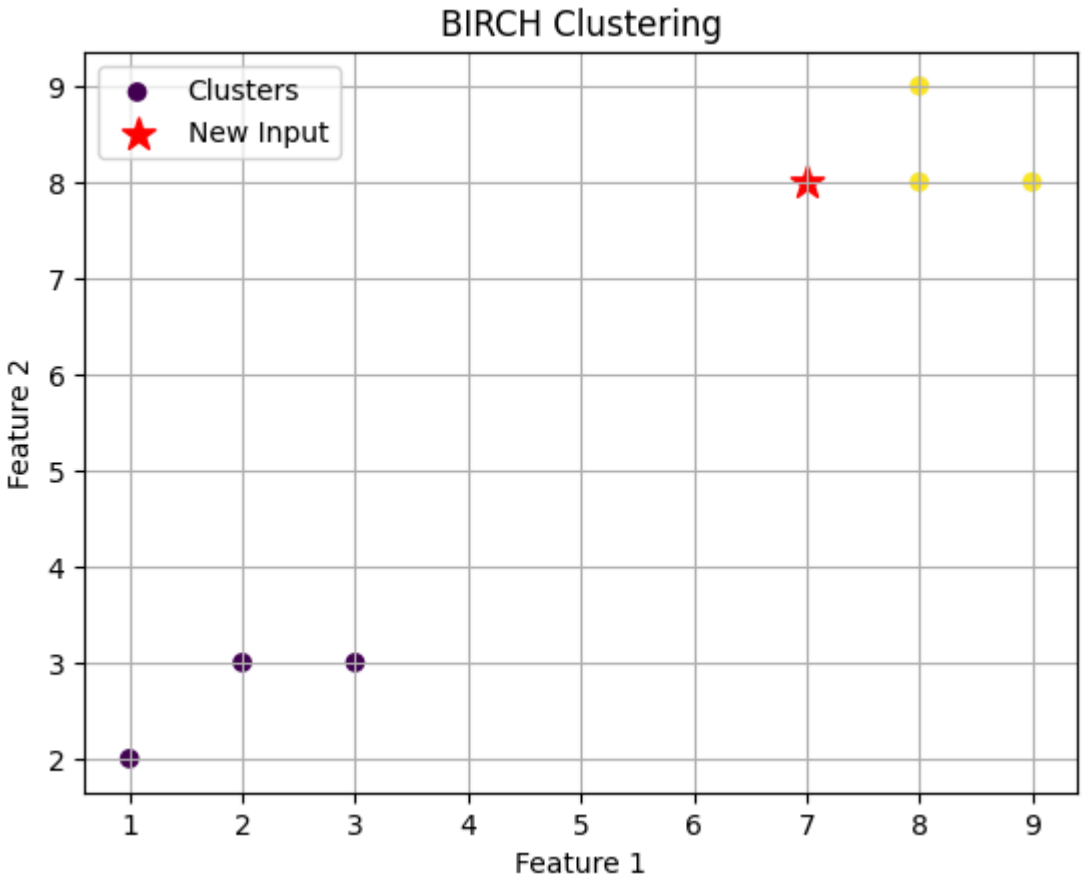
# Predict cluster
result = model.predict([[a, b]])
print("Cluster:", result[0])

# Scatter plot
plt.scatter([x[0] for x in X], [x[1] for x in X],
            c=model.labels_, cmap='viridis', label='Clusters')

plt.scatter(a, b, color='red', marker='*', s=150, label='New Input')

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("BIRCH Clustering")
plt.legend()
plt.grid()
plt.show()
```

```
Enter first value:  7
Enter second value:  8
Cluster: 1
```



Q15

In [4]:

```
from pyclustering.cluster.kmedoids import kmedoids
import matplotlib.pyplot as plt

# Training data
X = [[1,2], [2,3], [3,3],
     [8,8], [9,8], [8,9]]

# Initial medoids
initial = [0, 3]

# Create and train model
model = kmedoids(X, initial)
model.process()

# Get clusters and medoids
clusters = model.get_clusters()
medoids = model.get_medoids()

# User input
a = float(input("Enter first value: "))
b = float(input("Enter second value: "))

# Find nearest medoid
import math
dist = [math.dist([a,b], X[i]) for i in medoids]
print("Cluster:", dist.index(min(dist)))

# Scatter plot
for i, c in enumerate(clusters):
    plt.scatter([X[j][0] for j in c],
                [X[j][1] for j in c], label=f'Cluster {i}')

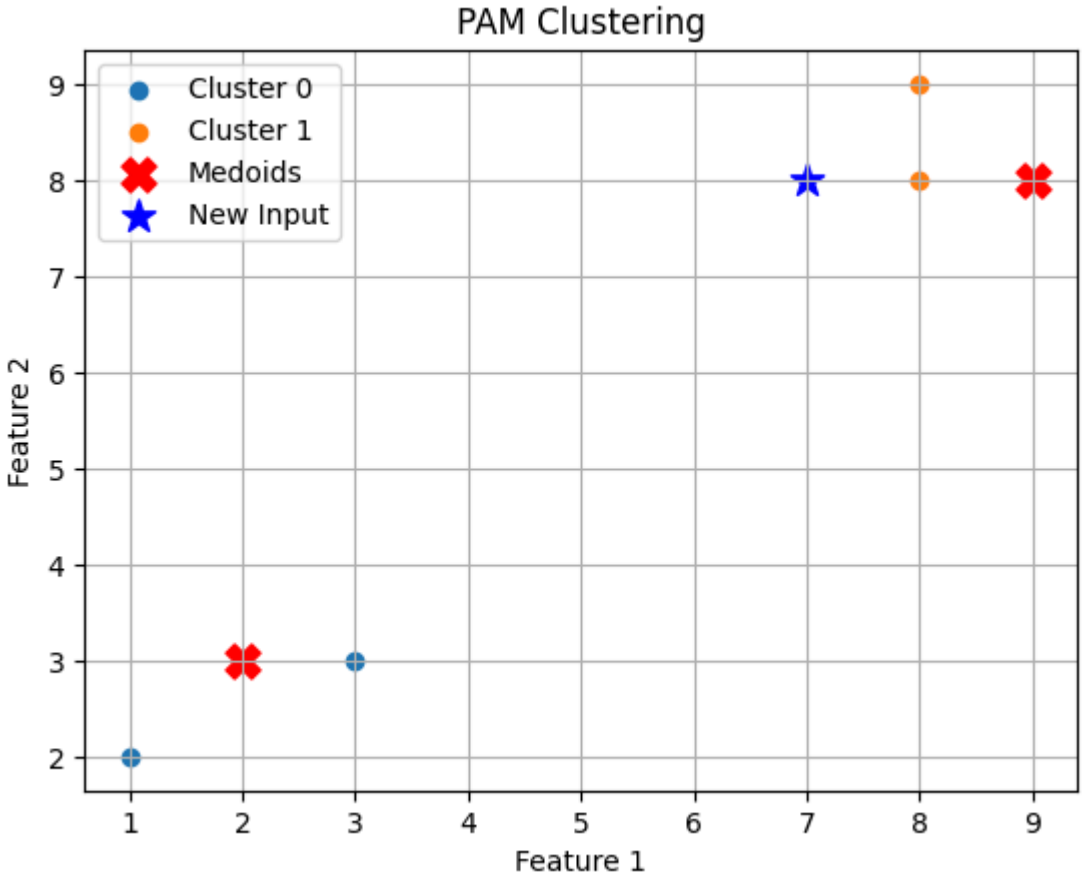
# Plot medoids
plt.scatter([X[i][0] for i in medoids],
            [X[i][1] for i in medoids],
            color='red', marker='X', s=150, label='Medoids')

# Plot user input
plt.scatter(a, b, color='blue', marker='*',
```

```
            s=150, label='New Input')

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("PAM Clustering")
plt.legend()
plt.grid()
plt.show()
```

```
Enter first value:  7
Enter second value:  8
Cluster: 1
```



Q16

In [5]:

```
from sklearn.cluster import DBSCAN
import numpy as np
import matplotlib.pyplot as plt

# Training data
X = [[1,2], [2,3], [3,3],
     [8,8], [9,8], [8,9]]

# Create DBSCAN model
model = DBSCAN(eps=2, min_samples=2)
model.fit(X)

# User input
a = float(input("Enter first value: "))
b = float(input("Enter second value: "))

# Find nearest cluster
labels = model.labels_
dist = np.linalg.norm(np.array(X) - [a,b], axis=1)
nearest = np.argmin(dist)

print("Cluster:", labels[nearest])

# Scatter plot
plt.scatter(np.array(X)[:,0], np.array(X)[:,1],
            c=labels, cmap='viridis', s=100)
```

```
plt.scatter(a, b, color='red', marker='*',
            s=150, label='New Input')

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("DBSCAN Clustering")
plt.legend()
plt.grid()
plt.show()
```

```
Enter first value:  7
Enter second value:  8
Cluster: 1
```

